

Cairo University
Faculty of Engineering
Computer Engineering Department

Data Structures and Algorithms

Sheet #3

Recursion

Part I: Exercises

1. Consider the following recursive algorithm:
 - a. Show the output and draw the sequence of recursive calls of the following:
 - i. func (10, 4)
 - ii. func (4, 4)
 - iii. func (4, 7)

Algorithm func (x, y)
//Input: 2 integers x and y
if x > y **return** -1
elseif x = y **return** 1
else return x * func (x + 1, y)

2. Consider the following recursive algorithm:
 - a. Show the output and draw the sequence of recursive calls of the following:
 - i. S(1)
 - ii. S(5)
 - b. What does the algorithm compute?

Algorithm S(n)
//Input: a positive integer n
if n = 1 **return** 1
else return S(n-1) + n*n*n

3. Consider the following recursive algorithm:
 - a. Show the output and draw the sequence of recursive calls of the following:
 - i. Riddle1([2, 3], 2)
 - ii. Riddle1([3, 5, 4, 1, 2], 5)
 - b. What does the algorithm compute?

Algorithm Riddle1(A, n)
//Input: An array A of real numbers of length n
if n = 1 **return** A[0]
 temp ← Riddle1(A, n-1)
if temp ≤ A[n - 1] **return** temp
else return A[n - 1]

4. Consider the following recursive algorithm:
 - a. Show the output and draw the sequence of recursive calls of the following:
 - i. Riddle2([2, 5, 3, 6], 0, 3)
 - ii. Riddle2([3, 5, 4, 1, 2], 0, 4)
 - b. What does the algorithm compute?

Algorithm Riddle2(A, first, last)
//Input: An array A of real numbers and 2 integers first and last
if first = last **return** A[first]
 mid = (first+last)/2
 temp1 ← Riddle2(A, first, mid)
 temp2 ← Riddle2(A, mid + 1, last)
if temp1 ≤ temp2 **return** temp1
else return temp2

5. Does it matter where you position your recursive call in your function?

Given two versions of recursive function that searches for a given value in a linked list; what is the difference between the two functions? Which one is better and why?

Test the two functions if linked list $L = [10] \rightarrow [12] \rightarrow [4] \rightarrow [7] \rightarrow [56] \rightarrow [9] \rightarrow \#$ and $val=12$.

```
Node* RecSearch1(Node *L, int val)
{
    if(L == NULL) return NULL;
    if(L->data == val) return L;

    return RecSearch1(L->Next, val);
}
```

```
Node* RecSearch2(Node *L, int val)
{
    if(L == NULL) return NULL;

    Node *P = RecSearch2(L->Next, val);
    if( P != NULL) return P;
    if(L->data == val) return L;
    else return NULL;
}
```

Part II: Problems

- Write a recursive function to get the sum of series $1+2+3+4+\dots+n$.
- Write a recursive function to add the first n elements of the series: $1+1/2+1/3+1/4+1/5+\dots+1/n$
- Write a recursive function that calculates and returns the sum of array elements.
- Write a recursive function to count number of occurrences of a value in an array.
- Write a recursive function that calculates and returns the length of a linked list.
- Write a recursive function to compute the sum of even numbers in a linked list.
- Write a recursive function to print a linked list in a forward order. What are the modifications to print it reversed?
- Write a recursive function to print numbers read from keyboard in a forward order. What are the modifications to print them reversed? (**Note:** don't store numbers in an array; just print)
- Compare the recursive solution of print reverse of problem 7 and 8 with the stack solution.
- Compare the recursive solution of print forward of problem 7 and 8 with the iterative solution.
- Write a recursive function that returns the index of the first occurrence of an element in array if found and -1 if not found. What are the modifications to return the index of last occurrence?
- Write a recursive function to check whether a linked list is sorted in an increasing order.
- Write a recursive function that checks whether a string is a substring of another string.
- Write a recursive function that converts a string of numerals to an integer. For example, "43567" will be converted to 43567. (**Hint:** $43567 = 4*10000+3*1000+5*100+6*10+7*1$)
- Write a recursive C++ function to get the length of a **circular** linked list (**Hint:** you should send the head of the original list too in each call; 2 inputs)
- Write a recursive function that reverses a linked list (without using new nodes). (**Hint:** make each next pointer points to previous. It's simple if you think recursively)
- Solve **palindrome** problem (stack problem 2, sheet 5) using recursion. (**Hint:** check the first character with the last character)
- Solve **Josephus** problem (problem 11, sheet 3) using recursion.